🚀 PUBLIC ALPHA V0.2.0-ALPHA.1

GitHub: wellparth/myploy

🚀 MyPloy — Tester & Architecture Playbook

A step-by-step technical guide explaining how MyPloy works under the hood, how to test each feature on your VPS, and why it provides a lightweight Dokploy alternative with a zero-daemon architecture.

Target OS: **Ubuntu 20.04/22.04/24.04 / Debian 11/12**

Engine: **Docker Compose v2**

Footprint: **< 35MB RAM (Zero Daemon)**

Security: **Origin Cloaking & Scrypt Auth**

💡 1. What is MyPloy & Why Does It Exist?

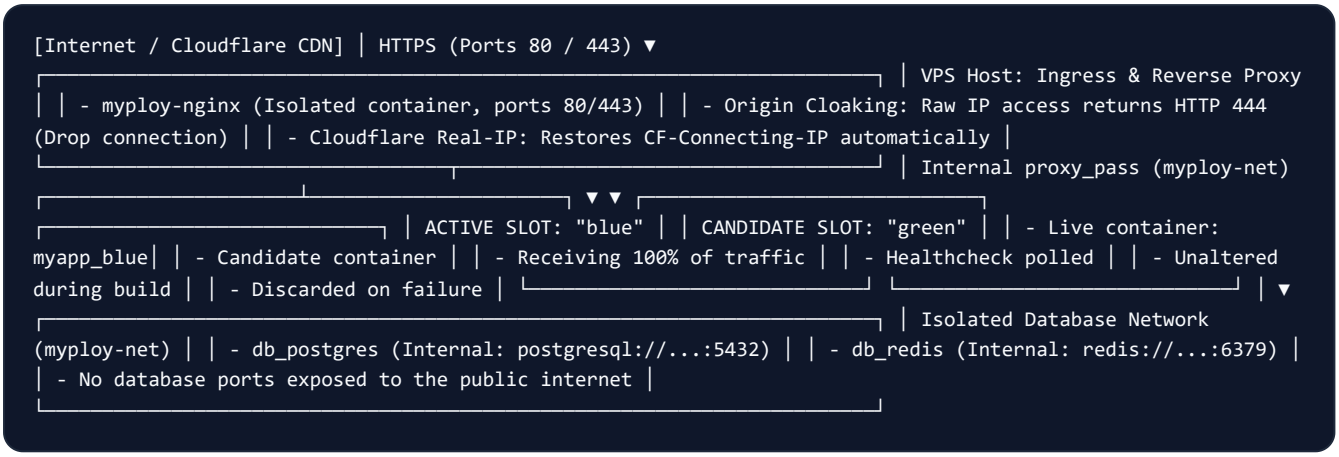
If you run containerized web applications on small VPS instances ($4–$10/mo from Hetzner, DigitalOcean, or Linode), existing solutions create an uncomfortable dilemma:

DEPLOYMENT MODEL	HOST RAM OVERHEAD	CONFIGURATION STYLE	CUTOVER METHOD
Dokploy / Coolify	⚠️ 1,000MB – 2,000MB (Heavy background daemon & DB)	Proprietary Web UI / Custom Schemas	Varies / Container swaps
Kamal (37signals)	✅ None (<20MB proxy)	Requires custom `deploy.yml` & Docker Hub/GHCR	Traefik container swap
Manual Bash Scripts	✅ None	Manual Nginx files, certbot cron, prone to typos	⚠️ Dropped packets during restarts
MyPloy (This Tool)	✅ **<35MB RAM (Shared Nginx proxy only)**	**Standard `docker-compose.yml` (Zero learning curve)**	**Atomic Zero-Downtime Blue/Green Switch**

ℹ️ **The "Zero-Daemon" Advantage:** MyPloy does not run a continuous heavy Node.js or Postgres daemon on your server. It connects securely over SSH, verifies the state, manages Docker containers via native CLI/Engine API, and exits immediately. When you want a UI, `myploy ui` serves an embedded dashboard locally or over a secured port on demand!

📐 2. Architectural Blueprint & Traffic Flow

Here is the lifecycle of how traffic traverses the MyPloy environment on your VPS:



🛡️ Origin Cloaking

Port scanners hitting your raw VPS IP get an immediate HTTP 444 response. Only requests with your authorized domain are routed to your app.

🔄 Blue/Green Slots

Host port bindings are automatically stripped from candidate compose files to prevent `port already in use` errors.

1 Step 1: Installing MyPloy (Alpha Testing)

Alpha testers can install MyPloy globally directly from GitHub or build from source:

```
# Option A: Direct Global Install from GitHub
npm install -g https://github.com/wellparth/myploy.git

# Option B: Clone & Link for local development
git clone https://github.com/wellparth/myploy.git
cd myploy
npm install && npm run build && npm link

# Verify installation:
myploy --version
# Expected Output: 0.2.0-alpha.1
```

2 Step 2: Automated SSH Key Pairing (Zero Errors)

SSH permission errors and key generation issues are the #1 point of failure in VPS tools. MyPloy completely automates this with high-security Ed25519 cryptography:

```
# Auto-generate a dedicated Ed25519 key and authorize it on your VPS in 1 command:
myploy ssh init --host 198.51.100.25 --user root

# Print your public key (ready to paste into Hetzner / DigitalOcean console):
myploy ssh public-key
```

✨ **Self-Healing Auth:** If you haven't authorized your key yet, `myploy setup` will automatically prompt for your VPS root password once, install the public key with strict `chmod 700 ~/.ssh` and `chmod 600 authorized_keys` permissions, and then switch to 100% passwordless SSH!

3 Step 3: Bootstrapping the Server (One-Time)

Turn any fresh VPS into an orchestrated deployment host:

```
myploy setup --host 198.51.100.25 --user root
```

What happens during setup?

1. Checks for Docker Engine and Docker Compose v2 (installs them via official Docker APT repos if missing).
2. Configures UFW firewall, strictly opening only ports `22` (SSH), `80` (HTTP), and `443` (HTTPS).
3. Creates the isolated Docker bridge network: `myploy-net`.
4. Deploys the lightweight shared `myploy-nginx` and `myploy-certbot` containers.

4 Step 4: Dokploy-Style Web Control Plane

Manage your entire VPS visually without leaving heavy daemons running:

```
# Launch dashboard on your workstation or VPS:
myploy ui --port 4000
```

Open `http://localhost:4000` (or `http://<vps-ip>:4000`):

🔐 Master Scrypt Auth

First launch prompts you to create a master password. Stored with Scrypt (64-byte derivation, unique salt) and timing-safe comparison.

🔍 Container Auto-Discovery

Automatically discovers all Docker containers running on your VPS—including manual standalone services—with start, stop, restart, and log viewing.

📊 Live Telemetry

Real-time CPU%, Memory usage, PID count, and server uptime gauges streamed via bounded JSON endpoints.

⚡ 1-Click Rollback

Visual button to instantly switch active traffic to the standby Blue/Green slot in <1 second.

5 Step 5: Zero-Downtime Blue/Green Deployments

Deploy your application repository containing a `docker-compose.yml`:

```
# 1. Generate myploy.json in your app directory:
myploy init

# 2. Deploy your app with atomic cutover:
myploy deploy
```

🔍 **The Verification Protocol:** MyPloy builds the candidate container alongside the live container. It polls `http://localhost:<port>/health`. If the app throws errors or fails to boot within the timeout, the candidate is destroyed and error logs are displayed. **Active production users experience zero interruption.**

6 Step 6: Instant 1-Second Rollbacks

If you deploy a bug, rollback immediately from either the CLI or Web UI:

```
myploy rollback
```

Because MyPloy keeps the previous slot alive in warm standby, rollback does not rebuild or restart containers—it executes an atomic Nginx upstream reload in ~0.2 seconds.

7 Step 7: 1-Click Managed Databases & Backups

Create databases on the internal `myploy-net` network without exposing ports to attackers:

```
# Provision PostgreSQL 16:
myploy db create postgres mydb

# Provision Redis 7, MySQL 8, or MongoDB 7:
myploy db create redis mycache
myploy db create mysql mystore
myploy db create mongo mydocs

# List all databases and internal connection strings:
myploy db list

# Create an immediate compressed gzip snapshot:
myploy db backup mydb
```

🛡️ **CWE-214 Hardening:** Passwords are never passed via `-e PASSWORD` on the command line. They are written to `chmod 600` environment files, preventing credential sniffing via `ps aux` on the VPS.

8 Step 8: Remote Git Deployments

Deploy directly from a public or private GitHub repository without keeping files locally:

```
myploy git https://github.com/myorg/myapp.git --branch main --domain api.example.com
```

9 Step 9: Alpha Testing Checklist & Bug Reporting

TEST SCENARIO	COMMAND TO RUN	WHAT TO VERIFY
1. SSH Keygen	`myploy ssh init --host <ip>`	Ed25519 key created, passwordless login works
2. VPS Setup	`myploy setup`	Docker, UFW, and Nginx containers running
3. Web UI	`myploy ui --port 4000`	Discovers all running VPS containers, Scrypt password works
4. App Deploy	`myploy deploy`	App accessible via domain, Blue/Green swap succeeds
5. Rollback	`myploy rollback`	Traffic reverts to previous slot in <1 second
6. Database	`myploy db create postgres mydb`	DB starts, backup produces `.sql.gz` file

🐛 **Found an edge case or bug?**
Please open an issue on GitHub: **github.com/wellparth/myploy/issues** using the **Alpha Bug Report** template. Include your VPS OS, Node.js version, and terminal logs.